Batch Geoprocessing and Validation of Monthly Snow Water Equivalent (SWE) Rasters Using ArcPy Conditional Logic in an ArcGIS Notebook

Introduction

For this coding project, I selected Project Track 2: Smart Batch Geoprocessing with Conditional Logic. This track focuses on creating a reproducible Python script using an ArcGIS Notebook that automates geoprocessing tasks while making decisions based on dataset properties. The goal of this project was to demonstrate the ability to structure a Python workflow that integrates ArcPy geoprocessing tools with fundamental programming principles such as loops, conditional statements, and user-defined functions.

The purpose of the script developed for this project is to automate the preprocessing of monthly Snow Water Equivalent (SWE) raster datasets for the Animas River watershed, a HU-8 watershed study area that has remained the focus of research in my graduate studies at NMSU. SWE is an important hydrologic variable used to estimate the amount of water stored in snowpack, which is particularly relevant in the western United States where seasonal snowpack contributes significantly to water resources, including agricultural and municipal storage and use. This script automates a workflow that validates raster datasets, projects them to a consistent coordinate system appropriate for spatial analysis, and clips them to a study area boundary. Automating this workflow reduces repetitive manual processing and ensures consistent outputs for further analysis steps.

Data and Workspace Description

The workspace used in this project contains monthly SWE raster datasets downloaded as GeoTIFF from the Climate Engine web application. A direct download from Oak Ridge National Laboratory (ORNL DAAC) was attempted multiple times, but the raster files were incomplete and easily compatible with a GIS workflow. These rasters represent monthly mean snow water equivalent values across a geographic context that includes the Animas River watershed, at a spatial resolution of 1km. The data are stored in a directory dedicated to raw raster inputs and have been named following a naming convention that includes ***watershedName_variable_YYYYMM_statisticalMeasure***.

The project also includes a polygon shapefile representing the study area extent, which defines the watershed boundary used for clipping the raster datasets. The shapefile uses

a custom Transverse Mercator coordinate system, which is used as the target projection for all raster outputs.

The datasets in this project consist primarily of:

- Raster datasets (GeoTIFF format) representing monthly SWE values
- Polygon feature class representing the watershed study area

Several assumptions were made when developing the script. First, the rasters are expected to be stored in TIFF format and to contain SWE data in their file name. Second, the rasters are assumed to have a valid spatial reference. If a raster does not have a defined coordinate system, it is flagged and skipped during processing. Finally, the script assumes that raster file names follow a consistent naming convention that includes the SWE identifier.

Workflow Explanation

The Python script automates the preprocessing workflow using several ArcPy functions and basic programming logic. The workflow begins by importing two libraries and setting the workspace environment, which defines the directory where the raw raster data are located. The script also enables overwriting of outputs and disables automatic addition of outputs to the ArcGIS Pro map to prevent clutter when processing large numbers of rasters.

Once the workspace is defined, the script uses the ArcPy function ListRasters() to generate a list of all raster datasets found within the defined workspace. This allows the script to dynamically identify and process all SWE raster files without requiring manual input or selection of each dataset name.

To evaluate each raster, the script uses the ArcPy Describe() function. This function extracts key metadata properties such as spatial reference name and raster format. These properties are used to perform rule-based validation checks before processing of the directory items begins.

A user-defined function, validate_raster(), is used to organize these checks. This function evaluates each raster to ensure it meets specific criteria before processing continues. For example, the function checks whether the raster has a valid spatial reference and verifies that file format is TIFF. If any required condition is not met, the script prints a warning message and skips the dataset. The script then enters a loop that iterates through each raster dataset identified in the workspace. The loop uses Python's enumerate() function to track progress and print status messages indicating which raster is currently being processed.

For rasters that pass validation, the script performs two geoprocessing operations. First, the raster is projected to match the coordinate system of the watershed boundary using the ProjectRaster tool. This ensures all datasets share a consistent spatial reference system appropriate for analysis steps. Next, the projected raster is clipped to the watershed area using the Clip tool with polygon clipping geometry. This step limits the raster extent to the study area, producing smaller and more focused datasets for analysis.

Output rasters are saved to designated folders using standardized file naming conventions. Each output raster is assigned a modified file name that indicates where it has been projected or clipped. Throughout the workflow, the script prints status messages that report the progress of the batch processing operation.

Results

The script successfully processed the SWE raster datasets using the automated workflow. Each raster dataset was validated, projected to the watershed coordinate system, and clipped to the study area boundary. The outputs were written to organized directories for projected rasters and clipped rasters.

The results demonstrate that the script effectively automates a repetitive GIS workflow. Instead of manually projecting and clipping each raster dataset, the script processes the entire dataset collection using a single automated routine. The total runtime for this script across all 123 rasters was approximately two minutes and 10 seconds, significantly reducing time and labor. This approach improves efficiency and ensures that all rasters are cleaned and processed using consistent parameters.

Example output files include the intermediate projected rasters and final watershed-specific clipped rasters. The clipped outputs display SWE values only within the watershed boundary, confirming that the clipping operation worked as intended. Additionally, printed messages in the notebook confirm that the script successfully iterated through each raster dataset and applied the processing steps in sequence, as shown in *Figure 1*.

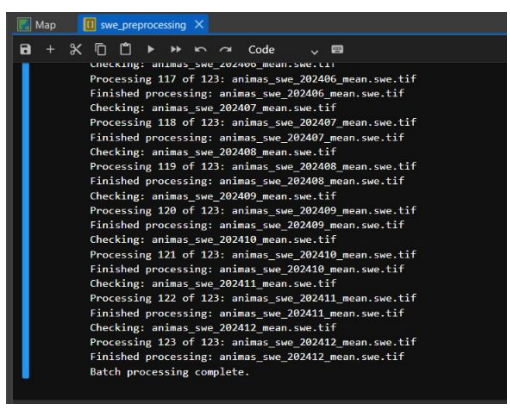

Figure 1: Processing and Finish Messages

Any datasets that failed validation checks would have been flagged with warning messages during execution, demonstrating the script's ability to enforce rule-based conditions during preprocessing. For a quick sanity check to ensure all functionalities of the script, a sample raster file was added to the directory that did not follow the naming convention but did fit the format type. The results in *Figure 2* show how the dataset item is caught and declared with the warning message.

```
Checking: animas_swe_202412_mean.swe.tif
Processing 123 of 124: animas_swe_202412_mean.swe.tif
Finished processing: animas_swe_202412_mean.swe.tif
Checking: conditionalFail_example.tif
WARNING: conditionalFail_example.tif does not follow SWE naming convention.
Batch processing complete.
```

Figure 2: Sanity check with conditional fail

After reviewing this functionality, I found that a warning message could be hard to read in a long list of processing tasks. In order to highlight these messages, I looked up Python documentation and found information about the display and HTML modules in the IPython.Display library. In order to update the script, I added an additional import statement and changed the warning statements to incorporate elements from these modules in a formatted string. The result is shown in *Figure 3*, with the warning statement now boldly standing out against the rest of the processing checks.

```
Processing 123 of 124: animas_swe_202412_mean.swe.tif
Finished processing: animas_swe_202412_mean.swe.tif
Checking: conditionalFail_example.tif
WARNING: conditionalFail_example.tif does not follow SWE naming convention.
Batch processing complete.
```

Figure 3: Newly formatted warning statement

Finally, outputs were pulled into a map in the ArcGIS Pro project, visually validating the processing steps. The raw data prior to processing is shown in *Figure 4*, while *Figure 5* shows the projected and clipped GeoTIFF symbolized more appropriately to show SWE. One further step was taken to manually process one of the outputs, using the SetNull tool to set a threshold in the Dec. 2024 raster for removing trace amounts of SWE (<2 mm) as shown in *Figure 6*. In other words, this raster shows the snowpack extent of values greater than 2mm.

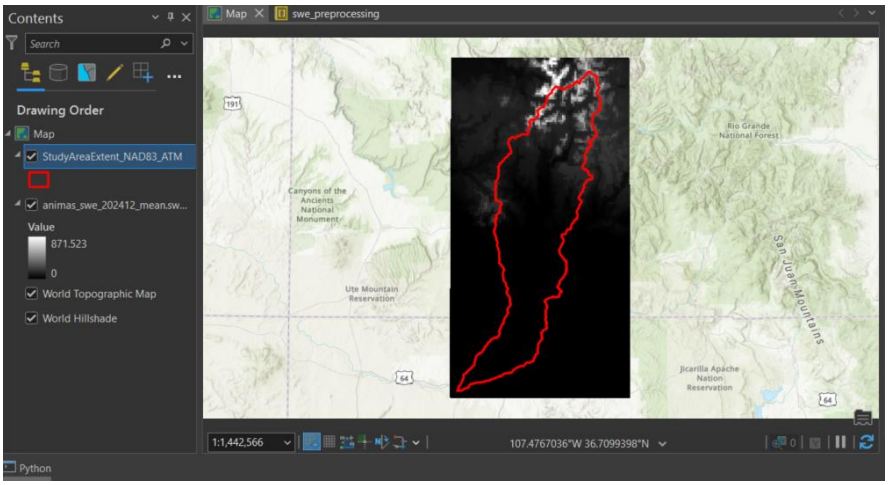

Figure 4: Raw raster before processing

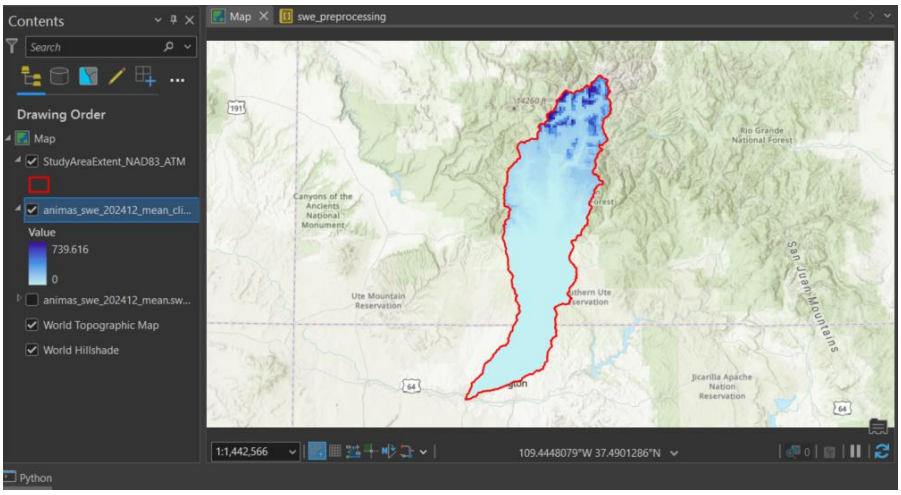

Figure 5: Projected, clipped, and symbolized SWE raster

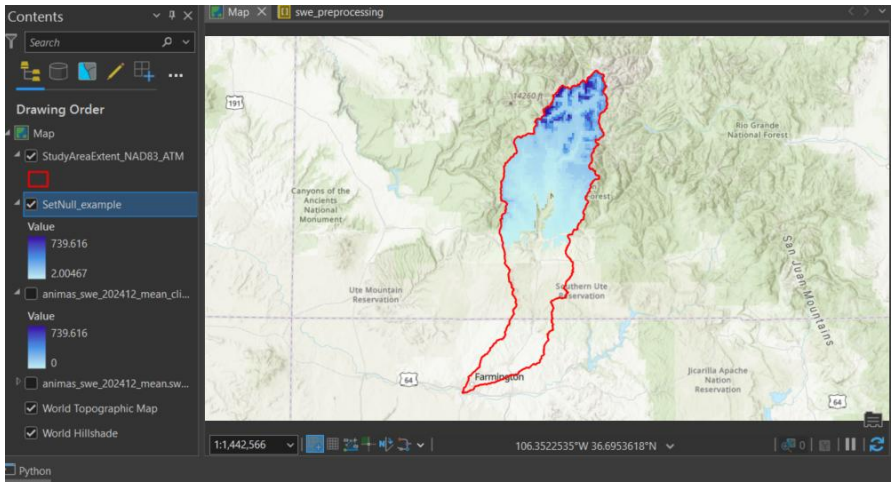

Figure 6: SWE extent of values > 2mm

Reflection

This project demonstrated the value of automating geospatial workflows using Python and ArcPy. One aspect that worked particularly well was the use of a user-defined function to validate raster datasets before processing. Organizing the validation checks in a function made the code easier to read and ensured the datasets were evaluated consistently.

One challenge encountered during development involved debugging syntax errors and confirming that file paths and workspace settings were correctly identified. Testing the script with a small subset of rasters helped identify and resolve these issues before running the full batch process. Printing intermediate messages and checking raster lists were useful strategies for verifying that the script was functioning as expected.

With additional time, the workflow could be expanded to include further analysis of the processed SWE datasets. For example, additional automation could be added to compute watershed-scale statistics or visualize seasonal snowpack patterns over time. The workflow could also be extended to automatically download new SWE datasets and incorporate them into a larger reproducible data processing pipeline. Both of these additional components will be added as time allows, creating a pipeline that takes raw data from an open source that is then processed and cleaned for analysis and visualization in a time-series animation with statistics and other insightful information stored in a table. While this workflow is currently done by taking advantage of an ArcGIS Pro environment, this could also be accomplished by targeting open-source software and updating the ArcPy statements appropriately.

Data Sources

Daymet Snow Water Equivalent (SWE)
Daymet Version 4 provides gridded meteorological data for North America at 1-km spatial resolution. Monthly mean SWE rasters used in this project were accessed through ClimateEngine.org for the period October 2014 through December 2024.

Animas River Watershed Boundary
The study area boundary was derived from the Hydrologic Unit Code 8 (HUC-8) watershed polygon for the Animas River from the U.S. Geological Survey National Hydrography Dataset Watershed Boundary Dataset (WBD).